

BDSEC CTF Writeup – Cold Start

1. Thông tin thử thách

- **Tên bài:** Cold Start
- **Thể loại:** Reverse Engineering
- **File:** cold_start.bdsec
- **Flag format:** BDSEC{...}

Khi chạy chương trình, ta nhận được giao diện yêu cầu nhập một activation seed:

```
=====
                        COLD START
=====

The system was powered down unexpectedly.

activation seed>
```

Nhập sai sẽ nhận được:

```
Boot sequence rejected.
```

Mục tiêu là tìm activation seed hợp lệ để khôi phục hệ thống và lấy flag.

2. Kiểm tra file ban đầu

Sử dụng lệnh `file`:

```
file cold_start.bdsec
```

Kết quả:

```
cold_start.bdsec: ELF 64-bit LSB pie executable, x86-64,
dynamically linked, stripped
```

Một số đặc điểm quan trọng:

- Binary ELF 64-bit.

- Kiến trúc x86-64.
- Có bật PIE.
- Binary đã bị stripped nên không còn tên hàm và tên biến.
- Chương trình liên kết động với libc.

Tiếp tục kiểm tra các chuỗi có trong binary:

```
strings -a cold_start.bdsec
```

Các chuỗi đáng chú ý:

```
COLD START
The system was powered down unexpectedly.
activation seed>
System restored.
Boot sequence rejected.
```

Flag không xuất hiện trực tiếp trong kết quả `strings`. Điều này cho thấy flag được lưu dưới dạng mã hóa và chỉ được giải mã sau khi nhập đúng seed.

3. Phân tích phần nhập liệu

Quan sát hàm `main` trong disassembler cho thấy chương trình sử dụng:

```
fgets(input, 0x40, stdin);
input[strcspn(input, "\r\n")] = '\0';
```

Sau đó chương trình kiểm tra độ dài:

```
if (strlen(input) != 6)
    reject();
```

Mỗi ký tự tiếp tục được kiểm tra bằng `isxdigit`, vì vậy input phải:

- Có đúng 6 ký tự.
- Chỉ gồm các ký tự hexadecimal: `0-9`, `a-f`, `A-F`.

Chuỗi sau đó được chuyển thành số bằng:

```
seed = strtoul(input, &endptr, 16);
```

Giá trị seed phải nằm trong phạm vi:

```
0x000000 <= seed <= 0xFFFFFFFF
```

Như vậy không gian tìm kiếm chỉ có:

```
224 = 16,777,216
```

giá trị.

Đây là một không gian đủ nhỏ để brute-force sau khi đã khôi phục chính xác thuật toán kiểm tra.

4. Khởi tạo trạng thái

Sau khi parse seed, chương trình khởi tạo ba biến trạng thái 32-bit:

```
a = rol32(seed * 0x9e3779b1, 7) ^ 0x1b873593;

d = (seed << 5)
  ^ (seed >> 3)
  ^ 0x811c9dc5;

s = seed ^ 0xa3c59ac3;
```

Ba biến đếm được khởi tạo bằng 0:

```
c1 = 0;
c2 = 0;
c3 = 0;
```

Trong đó `rol32` là phép xoay trái trên số nguyên 32-bit:

```
uint32_t rol32(uint32_t x, unsigned int n) {
    n &= 31;
    return (x << n) | (x >> ((32 - n) & 31));
}
```

Chương trình còn sử dụng một hàm trộn 32-bit:

```
uint32_t mix32(uint32_t x) {
    x ^= x >> 16;
    x *= 0x7feb352d;
    x ^= x >> 15;
    x *= 0x846ca68b;
    x ^= x >> 16;
    return x;
}
```

Ngoài ra, trong section `.rodata` có một bảng gồm 256 số nguyên 32-bit. Ta đặt tên bảng này là:

```
uint32_t T[256];
```

Bảng được sử dụng để tạo các giá trị phụ thuộc vào seed trong từng vòng lặp.

5. Thuật toán biến đổi 96 vòng

Chương trình chạy tổng cộng 96 vòng. Mỗi vòng tạo một giá trị trạng thái `out[i]`.

Pseudocode được khôi phục như sau:

```
for (uint32_t i = 0; i < 96; i++) {
    uint32_t idx =
        (uint8_t)(d ^ s ^ c1 ^ rol32(a, i));

    uint32_t table_value = T[idx];

    uint32_t rotation =
        (table_value >> 27) + 1;

    s = rol32(
        (c2 ^ seed) + s + table_value,
        rotation
    );

    uint32_t tmp = d + c3 + s;

    d = d * 0x01000193 + i;

    uint32_t idx2 =
```

```

        (uint8_t)((table_value >> 8) + idx + i);

    tmp += T[idx2];
    table_value += d;

    a ^= mix32(tmp);

    uint32_t x =
        rol32(a, (i % 13) + 1) ^ s;

    c1 += 0x045d9f3b;
    c2 += 0x27d4eb2d;
    c3 += 0x9e3779b9;

    d = x + table_value;

    out[i] = rol32(a, 11) ^ s ^ d;
}

```

Lưu ý rằng trong assembly, phép cộng:

```
c3 += 0x9e3779b9;
```

được compiler biểu diễn tương đương bằng:

```
sub r11d, 0x61c88647
```

Do số học 32-bit:

```
-0x61c88647 mod 2^32 = 0x9e3779b9
```

6. Ba điều kiện checksum

Sau 96 vòng biến đổi, chương trình lấy một số phần tử cụ thể trong mảng trạng thái để tạo ba checksum.

Checksum thứ nhất

```

chk1 = mix32(
    out[11]
    ^ out[83]
    ^ rol32(out[47], 9)
)

```

```
    ^ s  
);
```

Giá trị bắt buộc:

```
chk1 == 0x91e50c54
```

Checksum thứ hai

```
chk2 = mix32(  
    rol32(out[68], 13)  
    + out[23]  
    + a  
    + d  
);
```

Giá trị bắt buộc:

```
chk2 == 0xc2e4f8bd
```

Checksum thứ ba

```
chk3 =  
    (out[7] >> 5)  
    ^ (out[55] >> 11)  
    ^ out[91]  
    ^ seed;
```

Chương trình chỉ kiểm tra 16 bit thấp:

```
(uint16_t)chk3 == 0x9c8c
```

Seed chỉ được chấp nhận khi đồng thời thỏa mãn cả ba điều kiện.

7. Chiến thuật tìm seed

Không nên brute-force binary bằng cách khởi chạy chương trình hàng triệu lần vì chi phí tạo process sẽ rất lớn.

Thay vào đó, ta:

1. Khôi phục thuật toán kiểm tra từ assembly.
2. Viết lại toàn bộ checker bằng C.
3. Duyệt trực tiếp không gian `0x000000-0xFFFFFFFF`.
4. Dùng checksum thứ nhất và thứ ba làm điều kiện loại sớm.
5. Chỉ tính hoặc so sánh checksum cuối khi các điều kiện trước đã đúng.
6. Sử dụng OpenMP để chia không gian tìm kiếm cho nhiều CPU thread.

Phần kiểm tra chính:

```
static inline int check_seed(uint32_t seed) {
    uint32_t a =
        rol32(seed * 0x9e3779b1u, 7)
        ^ 0x1b873593u;

    uint32_t d =
        (seed << 5)
        ^ (seed >> 3)
        ^ 0x811c9dc5u;

    uint32_t s = seed ^ 0xa3c59ac3u;

    uint32_t c1 = 0;
    uint32_t c2 = 0;
    uint32_t c3 = 0;

    uint32_t v7, v11, v23, v47;
    uint32_t v55, v68, v83, v91;

    for (uint32_t i = 0; i < 96; i++) {
        uint32_t idx =
            (uint8_t)(d ^ s ^ c1 ^ rol32(a, i));

        uint32_t tv = T[idx];

        uint32_t rot = (tv >> 27) + 1;

        s = rol32(
            (c2 ^ seed) + s + tv,
            rot
        );

        uint32_t tmp = d + c3 + s;

        d = d * 0x01000193u + i;
    }
}
```

```

uint32_t idx2 =
    (uint8_t)((tv >> 8) + idx + i);

tmp += T[idx2];
tv += d;

a ^= mix32(tmp);

uint32_t x =
    rol32(a, (i % 13) + 1) ^ s;

c1 += 0x045d9f3bu;
c2 += 0x27d4eb2du;
c3 += 0x9e3779b9u;

d = x + tv;

uint32_t out =
    rol32(a, 11) ^ s ^ d;

switch (i) {
    case 7: v7 = out; break;
    case 11: v11 = out; break;
    case 23: v23 = out; break;
    case 47: v47 = out; break;
    case 55: v55 = out; break;
    case 68: v68 = out; break;
    case 83: v83 = out; break;
    case 91: v91 = out; break;
}
}

uint32_t chk1 = mix32(
    v11 ^ v83 ^ rol32(v47, 9) ^ s
);

if (chk1 != 0x91e50c54u)
    return 0;

uint32_t chk3 =
    (v7 >> 5)
    ^ (v55 >> 11)
    ^ v91
    ^ seed;

if ((uint16_t)chk3 != 0x9c8cu)
    return 0;

```

```

uint32_t chk2 = mix32(
    rol32(v68, 13) + v23 + a + d
);

return chk2 == 0xc2e4f8bdu;
}

```

Vòng brute-force đa luồng:

```

_Atomic uint32_t found = 0xffffffffu;

#pragma omp parallel for schedule(static)
for (uint32_t seed = 0;
    seed <= 0xffffffffu;
    seed++) {

    if (atomic_load_explicit(
        &found,
        memory_order_relaxed
    ) != 0xffffffffu) {
        continue;
    }

    if (check_seed(seed)) {
        atomic_store_explicit(
            &found,
            seed,
            memory_order_relaxed
        );
    }
}

```

Compile solver:

```

gcc -O3 -fopenmp cold_start_solver.c
-o cold_start_solver

```

Chạy solver:

```

./cold_start_solver

```

Kết quả:

```
FOUND 93C7A4
seed=93c7a4
a=9c97212f
d=3e4b4751
s=a99d1c32
chk1=91e50c54
chk2=c2e4f8bd
chk3low=9c8c
```

Ta thu được activation seed:

```
93C7A4
```

8. Kiểm tra trực tiếp với binary

Chạy chương trình với seed vừa tìm được:

```
printf '93C7A4\n' | ./cold_start.bdsec
```

Output:

```
=====
                        COLD START
=====

The system was powered down unexpectedly.

activation seed> System restored.
BDSEC{th3_k3y_w4s_s0m3wh3r3_1n_16_m1l110n}
```

Cả ba checksum của seed đều khớp:

```
chk1    = 0x91e50c54
chk2    = 0xc2e4f8bd
chk3low = 0x9c8c
```

9. Cơ chế giải mã flag

Flag được lưu trong `.rodata` dưới dạng một mảng 42 byte bắt đầu tại địa chỉ tương đối `0x2100`.

Sau khi seed vượt qua ba checksum, chương trình sử dụng:

- Activation seed.
- Các trạng thái `out[0..95]`.
- Byte mã hóa trong `.rodata`.
- Một biến khóa chạy bắt đầu bằng `0x5a`.

Hai chỉ số trạng thái được tạo theo công thức:

```
index1 = (11 + 17 * i) % 96;  
index2 = (7 + 29 * i) % 96;
```

Byte seed được lấy tuần hoàn:

```
seed_byte = (seed >> (8 * (i % 3))) & 0xff;
```

Dạng rút gọn của quá trình giải mã:

```
uint8_t key = 0x5a;  
  
for (int i = 0; i < 42; i++) {  
    uint32_t x = out[(11 + 17 * i) % 96];  
    uint32_t y = out[(7 + 29 * i) % 96];  
  
    uint8_t value =  
        encrypted_flag[i]  
        ^ (uint8_t)(37 * i)  
        ^ (uint8_t)x  
        ^ (uint8_t)(x >> 8)  
        ^ (uint8_t)(y >> 16)  
        ^ (uint8_t)(y >> 24)  
        ^ (uint8_t)(seed >> (8 * (i % 3)));  
  
    key ^= value;  
    putchar(key);  
}
```

Do flag phụ thuộc cả seed lẫn các trạng thái của 96 vòng biến đổi, việc chỉ trích xuất mảng mã hóa trong `.rodata` sẽ không đủ để đọc flag.

10. Kết luận

Thử thách sử dụng một activation seed 24-bit, được kiểm tra qua:

- 96 vòng biến đổi trạng thái 32-bit.
- Một bảng tra cứu 256 phần tử.
- Ba checksum độc lập.
- Một thuật toán giải mã flag phụ thuộc vào seed và trạng thái nội bộ.

Sau khi reverse checker và brute-force toàn bộ không gian 24-bit, seed hợp lệ được tìm thấy là:

```
93C7A4
```

Flag cuối cùng:

```
BDSEC{th3_k3y_w4s_s0m3wh3r3_1n_16_m1ll10n}
```